

[ACJC/2025/Q3]

3 Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 3.1
        Program code
        Output:
```

An 2-dimensional array is to be implemented using Object-Oriented Programming (OOP). It acts as a free list so that linked lists can be created. Each row in the array represents a node in the free list. The first column of each row contains the data, which is a string, and the second column contains the pointer, which is an integer and is the index of the row to which that row is pointing.

The linked lists are to store strings in alphabetical order.

Task 3.1

The `Array` class includes the following attributes.

- `container` is an array with `n` rows and 2 columns, where `n` is input by the user during initialization. The data in the first column are initialized as empty strings. To initialize the second column, each row points to the row below it, except for the final row, which points to `None`.
- `free` is the head pointer of the free list, which is initialized with a value of 0.

The `Array` class also includes the following methods.

- `get_data(index)` returns the data of the row at the given `index` in `container`.
- `set_data(index, new_data)` inserts `new_data` into the data column of the row at the given `index` in `container`.
- `get_pointer(index)` returns the pointer of the row at the given `index` in `container`.
- `set_pointer(index, new_pointer)` updates the pointer value of the row at the given `index` in `container` to `new_pointer`.

Write program code to define the `Array` class.

[4]

Task 3.2

The `LinkedList` class includes the following attributes:

- `array` is the `Array` object in which the linked list is actually stored, and is input by the user during initialization.
- `head` points to the head node of the linked list, and is initialized as `None`.

The `LinkedList` class also includes the following methods:

- `display()` displays the data in all the nodes of the linked list in order of the list.
- `add(new_data)` adds a node to the linked list with `new_data` as the data. The new node should be added so that the nodes in the linked list are in alphabetical order of their data. If there are no nodes available, it prints an appropriate error message.
- `delete(to_delete)` attempts to delete the node with the data `to_delete` from the linked list, and adds it back to the head of the free list. If the node cannot be found in the linked list, it prints an appropriate error message.

Write program code to define the `LinkedList` class.

[17]

[ACJC/2025/Q3 Cont']

Task 3.2

Write program code to:

- create an instance `MyArray` of `Array` with 10 rows;
- create two instances, `AnimalList` and `PlantList`, of `LinkedList` that store their data in `MyArray`;
- add the following nodes in the order given:
 - "CAT" to `AnimalList`
 - "ROSE" to `PlantList`
 - "GOAT" to `AnimalList`
 - "FERN" to `PlantList`
 - "HORSE" to `AnimalList`
 - "VINE" to `PlantList`
- attempt to delete "COW" from `AnimalList`;
- delete "ROSE" from `PlantList`;
- display the data in both `AnimalList` and `PlantList`.

Show

all

output.

[5]

Save your Jupyter Notebook for Task 3.